

Candidata: María Angélica Pérez Polanias
Perfil: QA Manual + Automatización

1. Análisis y Pruebas Manuales

Casos de Prueba

Escenario	Qué se quiere probar	Datos principales	Resultado esperado	Prioridad
1. Exitoso	Transferencia válida cumpliendo todas las reglas de negocio.	Cta Origen: Ahorros 4589 Banco: Banco Prueba Cta Destino: 987654321 Tipo: Ahorros Monto: \$150.000 COP.	Operación exitosa. Se genera ID de transacción, fecha, monto, destino y estado exitoso. Saldo actualizado a \$2.350.000 COP.	Alta
2. Saldo insuficiente	Intento de transferencia por monto superior al saldo disponible.	Cta Origen: Ahorros 4589 (Saldo: \$2.500.000 COP) Monto: \$3.000.000 COP.	Rechazo de la transacción. Mensaje "Saldo insuficiente". No se descuenta dinero ni se registra como exitosa.	Alta

3. Valor límite	Transferencia por el valor mínimo permitido (\$10.000 COP).	Cta Origen: Ahorros 4589 Monto: \$10.000 COP.	Transacción procesada correctamente. Saldo actualizado a \$2.490.000 COP.	Media
4. Dato obligatorio faltante	Envío del formulario sin ingresar la cuenta destino.	Cta Origen: Ahorros 4589 Cta Destino: [Vacío] Monto: \$50.000 COP.	El sistema bloquea el envío, resalta el campo en rojo y muestra "Campo obligatorio".	Alta
5. Seguridad	Intento de transferencia en sesión expirada/no autenticada.	Token/Sesión expirada o nula Monto: \$100.000 COP.	Sistema deniega la acción, redirige al Login y exige autenticación.	Alta
6. Crítico de negocio	Transferencia con valor negativo o alfanumérico.	Cta Origen: Ahorros 4589 Monto: -\$50.000 o ABC.	Validación en cliente/servidor bloquea la transacción. Muestra "Monto inválido".	Alta

Análisis de Defecto: Falso Fallo con Descuento en Saldo

- **Severidad: Crítica / Bloqueante.** Afecta directamente la integridad financiera del usuario. Al debitar saldo sin completar la transacción, genera un problema real de pérdida de dinero.
- **Prioridad: Alta.** Afecta un flujo clave del negocio y debe corregirse de inmediato.
- **Riesgo de Negocio:** Pérdida de confianza del cliente, potencial impacto legal o regulatorio, aumento de tickets en soporte/atención al cliente y descalces contables en la plataforma.

Criterios de Aceptación y Validaciones para el Cierre

Antes de dar por resuelto este defecto, es necesario realizar y aprobar las siguientes comprobaciones:

1. **Auditoría de Logs (Backend):** Revisar la trazabilidad en los microservicios de transferencias para confirmar cómo se están gestionando los timeouts y validar que no existan transacciones no atómicas.
2. **Re-Testing:** Reproducir la prueba bajo escenarios adversos simulando micro-cortes de red y timeouts en la comunicación entre servicios.
3. **Garantía de Atomicidad y Rollback:** Asegurar que ante cualquier excepción o fallo técnico durante el flujo, la transacción ejecute un rollback completo y el dinero permanezca en la cuenta de origen.
4. **Consistencia en Base de Datos:** Verificar la coherencia entre el estado final de la transacción (FAILED / REJECTED) y el saldo real reflejado en la cuenta del usuario.

2. Playwright - Automatización web

Repositorio del proyecto: https://github.com/angelicaQa-ui/Pruebas_Nexos

3. Postman – API Test

(Nota: La colección exportada de Postman con los requests y los scripts de validación se encuentra adjunta en el repositorio de GitHub del punto anterior).

¿La prueba debe considerarse exitosa? No.

Explicación: En QA no podemos confiarnos solo de que la API responda un 200 OK o un "SUCCESS". Si la regla de negocio principal indica que el dinero se debe descontar y el saldo sigue igual, tenemos un problema real. Esto significa que hay una inconsistencia: la transacción no se completó en la base de datos o algún proceso asíncrono en el backend falló de forma silenciosa.

¿Qué validaría? Antes de aprobar la prueba, revisaría directamente en la base de datos en qué estado quedó esa transacción y apoyaría al equipo revisando los logs del servidor para entender por qué el servicio de saldos no aplicó el descuento.

4. Appium + JMeter

Nota: Mi experiencia práctica se ha centrado principalmente en pruebas manuales, automatización web y APIs. Aunque no he tenido la oportunidad de implementar Appium o JMeter en un entorno real de producción, realicé una investigación autodidacta enfocada en buenas prácticas para poder resolver este reto. Así es como abordaría ambos escenarios:

A. Appium

- **¿Cómo automatizaría el flujo?** Aplicaría el patrón Page Object Model (POM) tal como lo hago en web. El flujo sería: ingresar credenciales en el login, tocar el botón de ingreso, esperar a que cargue el dashboard, seleccionar la cuenta y capturar el texto del saldo.
- **Elemento para validar el login:** Buscaría un elemento único de la pantalla principal que confirme que la sesión inició correctamente, como un mensaje de bienvenida o el botón del menú lateral.
- **Assertion sobre el saldo:** Capturaría el texto del elemento en pantalla y haría un assert básico. Ejemplo:
`expect(textoSaldo).toContain("$2.500.000")`
- **Estrategia de localización:** Según lo que investigué, priorizaría totalmente el **Accessibility ID** (que equivale a `content-desc` en Android y `accessibilityIdentifier` en iOS). Es la mejor práctica recomendada porque sirve para ambas plataformas, es más rápido y no se rompe si cambian el diseño visual de la app, a diferencia de XPath.

B. JMeter

- **¿Considera que el servicio presenta un comportamiento adecuado?** No, basándome en los conceptos de pruebas de rendimiento, este comportamiento es deficiente para un servicio financiero.
- **¿Qué indicador analizaría primero y por qué?** Analizaría primero el **Percentil 95 (P95) de 4.200 ms**. Un P95 de 4.2 segundos significa que el 5% de los usuarios están sufriendo una demora muy alta solo para consultar su saldo. Revisaría este indicador porque lo más probable es que esta extrema lentitud (cuellos de botella o base de datos saturada) sea la causa raíz que está provocando los timeouts y generando ese **2% de errores**, lo cual también es un valor crítico.

5. Conceptos ISTQB Aplicados

1. Diferencia entre Severidad y Prioridad

- **Severidad:** Es el impacto técnico que tiene el defecto sobre el funcionamiento del sistema (qué tanto "rompe" la aplicación). Normalmente la define el equipo de QA.
- **Prioridad:** Es la urgencia temporal con la que el negocio o el cliente necesita que se solucione ese defecto. Normalmente la define el Product Owner.
- *Ejemplo en el contexto bancario:* Si en la pantalla de transferencias el logo del banco sale con los colores invertidos o un error ortográfico, la severidad es baja (porque el usuario aún puede transferir dinero sin problemas técnicos), pero la prioridad es altísima (porque afecta la imagen, la confianza del cliente y el negocio exige que se arregle hoy mismo).

2. Imposibilidad de probar todas las combinaciones posibles

- **Análisis:** Aquí aplica el principio de ISTQB que dicta que *"la prueba exhaustiva es imposible"*. Probar absolutamente cada número de cuenta, todos los montos posibles y todos los bancos nos tomaría tiempo infinito.
- **Cómo lo abordaría:** Para asegurar la calidad sin probar todo, utilizo técnicas de diseño de pruebas como Partición de Equivalencia y Análisis de Valores Límite. Esto me permite agrupar los datos en categorías representativas (ej. un caso con saldo exacto, un caso que lo supere por \$1, un caso negativo) y probar solo los límites, logrando una cobertura óptima en el menor tiempo posible.

3. Casos de regresión que llevan meses ejecutándose sin fallos

- **¿Demuestra que no hay defectos?** No. Aquí aplica el principio de la *"Paradoja del pesticida"*.
- **Explicación:** Si ejecutamos exactamente los mismos casos de prueba una y otra vez durante meses, llegará un punto en que dejarán de encontrar defectos nuevos, porque esa parte del código ya está estabilizada. Que los tests pasen en verde no significa que el software no tenga bugs, sino que los bugs están escondidos en escenarios que no estamos evaluando. La solución es revisar, rotar y actualizar los casos de prueba constantemente.

4. Diferencia entre Retesting y Regression Testing

- **Retesting (Pruebas de confirmación):** Es ir directamente al grano. Cuando un desarrollador repara un bug que yo reporté, vuelvo a ejecutar únicamente los pasos de ese defecto específico para confirmar que el problema ya no ocurre.
- **Regression Testing (Pruebas de regresión):** Es mirar el panorama completo. Consiste en ejecutar una suite de pruebas (idealmente automatizadas) sobre el resto del sistema que no fue modificado, para asegurar que el nuevo arreglo del desarrollador no haya dañado accidentalmente otra funcionalidad que antes trabajaba bien.